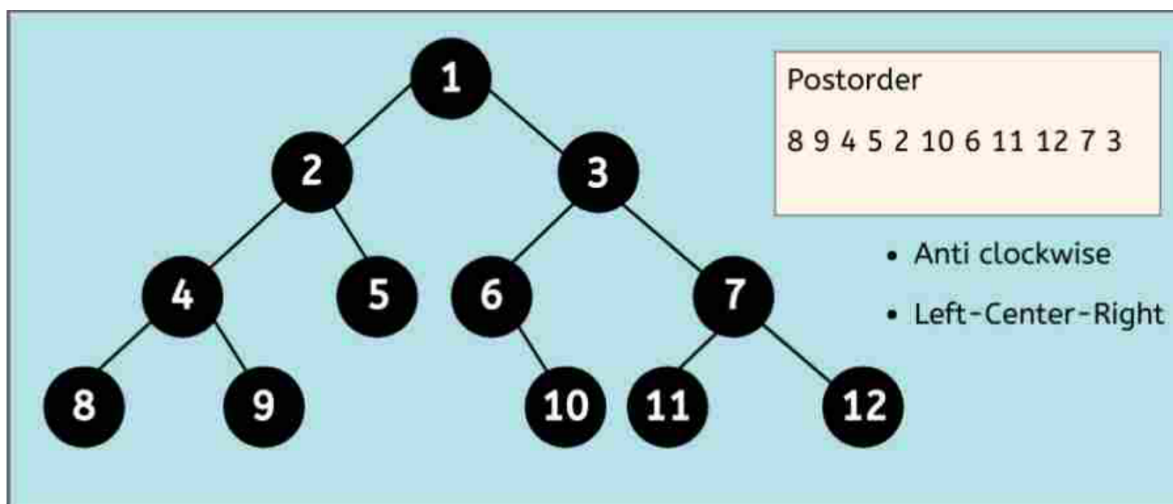


TREE 1

PROBLEM 4

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 64
Question description Darsh, Ratik, Swathy are good friends, They are studying Pre-final year B.E. in reputed institution. Swathy's uncle was a DS teacher in school of computing technologies. He asks to make an application for identifying postorder Traversal of the Tree You have to do the work for the development of the code of their thinking. Functional Description					



Constraints

$T \leq 100$

$1 \leq n \leq 1000$

$0 \leq k < n$

Input Format

The first line of input consists of the number of test cases T . Then T lines follow each describing a test case. The first line of each test case consists of a number n . Then follow n lines each describing the friendship of a student. Each line starts with a number k telling the number of friends the i th person has. Then follow k integers separated by space which tells the index (1 based) of his friends. All these integers on the i th line are greater than i itself. (See sample for clarity).

Output Format

For each test case, if it is possible to divide the class into teams, then print a line of L's and R's denoting which team that student is a part of. If it is impossible to divide the class then print -1 instead. If there are multiple solutions print the one that would come first in the dictionary.

INPUT (STRING) 7 98 78 58 75 130 110 150 EXPECTED OUTPUT 58 75 78 110 150 130 98	INPUT (STRING) 9 50 23 56 89 43 75 130 110 150 EXPECTED OUTPUT 43 23 75 110 150 130 89 56 50
Mandatory Test Cases	
Test Case 1 KEYWORD struct node	Test Case 2 KEYWORD struct node *left,*right;
Test Case 3 KEYWORD struct node* newNode(int item)	
Test Case 4 KEYWORD void postorder(struct node* root)	
Complexity Test Cases	
Test Case 1 CYCLOMATIC COMPLEXITY 1	Test Case 2 TOKEN COUNT 290
	Test Case 3 NLOC 50

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node* make_node(int v) {
    Node* p = (Node*)malloc(sizeof(Node));
    p->value = v;
    p->left = p->right = NULL;
    return p;
}

Node* append_bst(Node* root, int v) {
    if (!root) return make_node(v);
    if (v < root->value) root->left = append_bst(root->left, v);
    else if (v > root->value) root->right = append_bst(root->right, v);
    return root;
}

void display_postorder(Node* root) {
    if (!root) return;
    display_postorder(root->left);
    display_postorder(root->right);
    printf("%d ", root->value);
}

int main() {
    int total_nodes;
    if (scanf("%d", &total_nodes) != 1) return 0;

    Node* root = NULL;
    for (int i = 0; i < total_nodes; i++) {
        int item;
        scanf("%d", &item);
        root = append_bst(root, item);
    }
    display_postorder(root);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
7
90 70 50 75 130 110 150
50 75 70 110 150 130 90
```

```
9
50 23 56 89 43 75 130 110 150
43 23 75 110 150 130 89 56 50
```